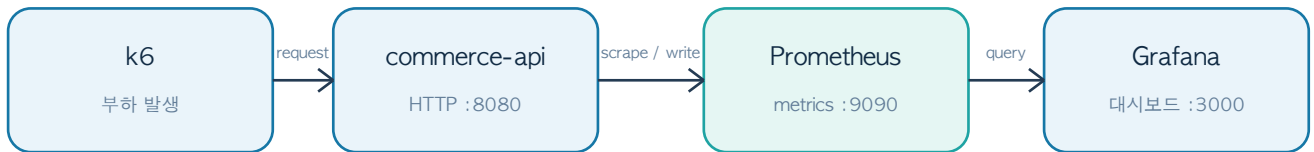


Loopers 이커머스 성능 실험 워크북

p95 - p99 - k6 - Prometheus - Grafana
측정부터 병목 해석까지



대상: Spring Boot 서비스를 실행해 본 주니어-미들 백엔드 개발자
환경: Java 21 / Spring Boot 3.4.4 / k6 / Prometheus / Grafana

학습 안내와 목차

이 워크북은 한 번 읽고 끝내는 설명서가 아니라, 명령을 실행하고 Grafana 화면에서 근거를 찾도록 구성했습니다. 측정하지 않은 숫자는 예시로 표시했습니다.

가정: 첫 실험은 상품 목록 조회입니다. 로컬 한 대의 결과를 운영 용량으로 직접 환산하지 않습니다.

학습 목표

- 평균과 p50-p95-p99의 차이를 설명한다.
- 목표 RPS와 지연시간을 함께 보고 포화 지점을 찾는다.
- k6 사용자 지연과 Spring 서버 지연을 구분한다.
- Tomcat, HikariCP, JVM 지표로 병목 가설을 세운다.
- 성능과 함께 재고-잔액-이벤트 정합성을 검증한다.

목차

학습 안내와 목차	2
학습 목표	2
목차	3
0. 실험의 전체 흐름	5
왜 중요한가	5
요청 여정	5
성공 기준	5
1. p95와 p99를 정확히 읽기	6
정신 모델	6
계산 예	6
표본 수와 히스토그램	6
확인 문제	6
2. 프로젝트를 실행하고 첫 측정 만들기	7
2.1 기능 기준선	7
2.2 인프라와 애플리케이션	7
2.3 데이터 적재	7
2.4 모니터링	7
3. k6 부하 프로파일과 합격 기준	8
도착률 기반 모델	8
첫 smoke	8
예시 threshold - 실제 SLO로 교체	8
4. 정규화-비정규화-Redis 비교	9
시나리오	9
공정 비교 조건	9
Redis cold-warm	9
결과 표	9
5. Grafana로 병목 좁히기	10
읽는 순서	10
증상과 가설	10
PromQL - 성공 요청 p95	10
6. 대용량 트래픽 측정 지도	11
네 가지 핵심 신호	11
계층별 확인	11
7. 프로젝트 전용 후속 실험	12
A. Hot SKU 재고 경합	12

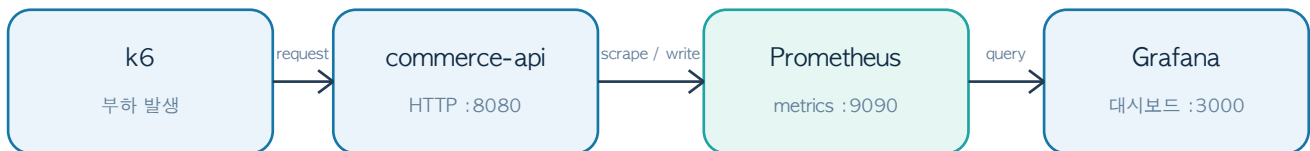
B. 포인트 동시 충전	12
C. PG 지연과 회로 차단기	12
D. 주문 -> Kafka -> streamer	12
내 실험 가설	12
8. 실패 진단 클리닉	13
사례 1 - p99만 급증	13
사례 2 - 목표 RPS 미달	13
사례 3 - Redis가 DB보다 느림	13
레이어 진단 문제	13
9. 실험 기록지	14
환경과 입력	14
측정 결과	14
가설 - 변경 - 결론	14
10. 종합 연습	15
11. 정답과 판단 근거	16
전이 질문	16
12. 공식 자료와 범위	17
QA 체크	17

의도적 생략: 분산 부하 발생기, DB-Redis-Kafka exporter, 분산 추적, 운영 SLO와 비용 산정.

0. 실험의 전체 흐름

왜 중요한가

성능 수치 하나만 보면 원인을 알 수 없습니다. k6는 사용자가 기다린 전체 시간을 측정하고, Spring Actuator는 서버 내부 처리와 자원 상태를 측정합니다. 두 시간측을 맞춰야 병목이 어느 경계에 있는지 좁힐 수 있습니다.



요청 여정

순서	구간	주요 증거
1	k6가 일정 RPS로 HTTP 요청	actual RPS, dropped iterations
2	Tomcat이 요청을 워커 스레드에 배정	busy / max threads
3	애플리케이션이 DB-Redis-Kafka 사용	Hikari pending, cache, lag
4	Actuator가 서버 메트릭 노출	http.server.requests histogram
5	Prometheus 수집-저장	targets, time series
6	Grafana에서 같은 시간축 비교	p95-p99, 오류, 포화

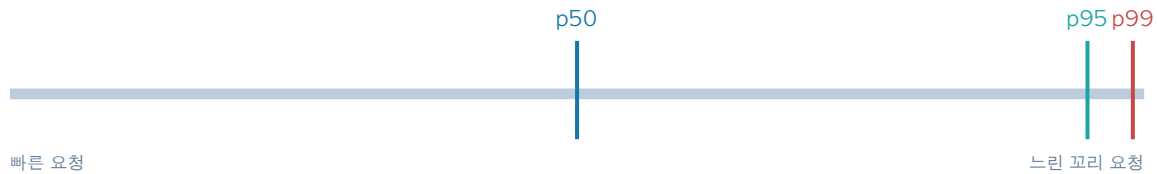
성공 기준

- 기능 테스트가 먼저 통과한다.
- 목표 RPS를 만들고 dropped iterations가 0이다.
- 기능 check와 HTTP 오류 기준을 통과한다.
- p95-p99가 SLO 안에 있고 포화 지표가 지속 증가하지 않는다.
- 최종 DB 상태가 비즈니스 불변식을 만족한다.

1. p95와 p99를 정확히 읽기

정신 모델

성공 요청 10,000개를 빠른 순서로 놓으면 p95는 약 9,500개가 그 값 이하, p99는 약 9,900개가 그 값 이하라는 뜻입니다. p99는 최댓값이 아니라 상위 1%가 시작되는 경계입니다.



비유: 평균 이동 시간은 하루 전체 교통의 대략적인 모습이고, p99는 혼잡한 순간에 겪을 수 있는 긴 이동 시간입니다. 한계: p99는 단 하나의 최악 상황이 아닙니다.

계산 예

$$\text{경계보다 느린 요청/초} = \text{RPS} \times (1 - P / 100)$$

RPS = 초당 요청 수 [요청/초]

P = 백분위 숫자 (p99이면 99)

합성 예시: 10,000 RPS에서 p99가 1초라면 매초 약 100개 요청이 1초보다 느릴 수 있습니다.

표본 수와 히스토그램

총 50건으로 계산한 p99는 가장 느린 한두 요청에 크게 흔들립니다. 비교 구간마다 성공 요청 10,000건 이상을 확보하면 상위 1%에 약 100개 표본이 생겨 해석이 쉬워집니다. 같은 조건은 최소 세 번 반복합니다.

서버 여러 대의 p99는 각 서버가 미리 계산한 p99를 평균 내면 안 됩니다. 요청시간 bucket을 합친 뒤 histogram_quantile()로 계산합니다.

확인 문제

평균 80ms, p95 200ms, p99 2s라면 평균만 보고 정상이라고 판단할 수 없는 이유를 적으세요.

2. 프로젝트를 실행하고 첫 측정 만들기

2.1 기능 기준선

```
mise exec java@21.0.2 -- ./gradlew build
```

기대 신호: BUILD SUCCESSFUL. Testcontainers가 MySQL-Redis를 사용하므로 Docker Desktop이 켜져 있어야 합니다.

2.2 인프라와 애플리케이션

```
docker compose -f docker/infra-compose.yml up -d
```

```
mise exec java@21.0.2 -- ./gradlew :apps:commerce-api:bootRun \
--args='--spring.jpa.show-sql=false'
```

```
curl -fsS http://localhost:8081/actuator/health
```

local 프로파일의 SQL 출력은 측정값을 왜곡할 수 있어 끕니다. 앱 시작 시 ddl-auto=create가 스키마를 다시 만들 수 있으므로 데이터를 넣기 전에 시작합니다.

2.3 데이터 적재

```
docker compose -f docker/infra-compose.yml exec -T mysql \
mysql -uapplication -papplication loopers < sql/data_setting.sql
```

기대 규모: 브랜드 500, 상품 300,000, 회원 10,000, 좋아요 299,999. 이 SQL은 실험 테이블 데이터를 지우므로 local 새 스키마에서만 실행합니다.

2.4 모니터링

```
docker compose -f docker/monitoring-compose.yml up -d
```

```
# Prometheus targets
open http://localhost:9090/targets
```

```
# Grafana: admin / admin
open http://localhost:3000
```

Grafana에서 Dashboards > Loopers > Loopers Performance Lab을 엽니다.

3. k6 부하 프로파일과 합격 기준

도착률 기반 모델

서버가 느려져도 예정된 RPS를 계속 만들도록 arrival-rate 실행기를 사용합니다. 필요한 가상 사용자를 만들지 못하면 dropped_iterations가 증가합니다. 서버 포화와 부하 발생기 포화를 함께 확인해야 합니다.

프로파일	확인 질문	기본 형태
smoke	경로-메트릭이 연결되는가?	2 RPS, 30초
load	예상 부하를 지속 처리하는가?	50 RPS, 5분
stress	어디서 무너지는가?	25 -> 200 RPS
spike	폭증 뒤 회복하는가?	20 -> 500 -> 20 RPS
soak	누수-지연이 누적되는가?	30 RPS, 30분

첫 smoke

```
K6_PROMETHEUS_RW_SERVER_URL=http://localhost:9090/api/v1/write \
K6_PROMETHEUS_RW_TREND_STATS=p(50),p(95),p(99),max \
k6 run -o experimental-prometheus-rw \
-e PROFILE=smoke -e VARIANT=denormalized \
-e TEST_ID=local-smoke \
k6/product-query-performance.js
```

예시 threshold - 실제 SLO로 교체

검사	예시 기준	의미
p95	< 500ms	95% 요청의 경계
p99	< 1000ms	느린 꼬리 요청의 경계
오류율	<= 1%	상태-응답 형식 검사 실패 포함
checks	>= 99%	기능 응답 검증
dropped	= 0	예정한 부하 생성 성공

Threshold 실패는 k6를 0이 아닌 종료 코드로 끝냅니다. Grafana는 악화 시점을 분석하고, 최종 합격 판정은 k6 threshold와 비즈니스 불변식을 함께 사용합니다.

4. 정규화-비정규화-Redis 비교

시나리오

variant	경로	주요 작업
normalized	/api/v1/products	product_like 조인- 집계
denormalized	/denormalization	product.like_count 읽기
redis	/redis	warm hit 또는 DB fallback

공정 비교 조건

- 같은 데이터 스냅샷과 인덱스
- 같은 BRAND_IDS, PAGE, SIZE, SORT
- 같은 RPS-실행 시간-워밍업 제외 구간
- 한 번에 변수 하나만 변경
- 성능 전 응답 내용 동일성 확인

Redis cold-warm

목록 캐시는 page 0-1만 저장합니다. 기존 스크립트의 page=100과 매 요청마다 달라지는 brandIds는 cache hit 실험이 아닙니다. 새 스크립트는 고정 브랜드와 page 0-1을 기본값으로 사용하고 setup에서 warm-up합니다.

현재 캐시 key에는 SIZE가 없습니다. 같은 조건에 서로 다른 size를 사용하면 잘못된 캐시 응답을 재사용할 수 있습니다. 이는 성능보다 먼저 확인할 정확성 문제입니다.

결과 표

지표	normalized	denormalized	redis warm
actual RPS			
p50 / p95 / p99			
오류 / dropped			
CPU 최대			
Hikari pending 최대			
응답 동일성			

5. Grafana로 병목 좁히기

읽는 순서

- k6: 목표 RPS, actual RPS, dropped, 오류, p95-p99
- HTTP: URI별 서버 p50-p95-p99와 5xx
- Tomcat: busy threads / max threads
- HikariCP: active-pending-max connections
- JVM: process CPU, heap, GC pause, live threads

증상과 가설

증상	함께 보이는 신호	우선 의심
k6 p99만 높음	서버 p99는 낮음	부하 발생기-네트워크-앞단 큐
서버 p99 증가	Hikari pending 증가	느린 쿼리-긴 트랜잭션-DB lock
서버 p99 증가	Tomcat busy=max	동기 외부 호출-워커 포화
p99와 CPU 증가	DB pending 낮음	계산-직렬화-GC-로그
오류 증가, p99 감소	빠른 4xx-5xx	실패가 수치를 좋아 보이게 함

PromQL - 성공 요청 p95

```

histogram_quantile(
  0.95,
  sum by (le, uri) (
    rate(http_server_requests_seconds_bucket{
      application="commerce-api", status=~"2.."
    }(5m))
  )
)

```

p99는 0.95를 0.99로 바꿉니다. histogram_quantile() 전에 bucket을 합칠 때 le label을 반드시 보존합니다.

6. 대용량 트래픽 측정 지도

네 가지 핵심 신호

범주	최소 지표	질문
지연시간	p50-p95-p99-timeout	느린 사용자는 얼마나 많은가?
트래픽	목표-실제 RPS, 동시 요청	필요한 부하를 처리했는가?
오류	HTTP, check, retry	빠른 실패도 잡았는가?
포화	CPU-thread-pool-queue	어떤 자원이 먼저 찼는가?

계층별 확인

계층	지표	경고 신호
k6	RPS, p99, failed, dropped	목표 RPS 미달
HTTP	URI별 RPS-p99-5xx	일부 URI만 악화
Tomcat	busy-max-reject	busy가 max에 고정
JVM	CPU-heap-GC-thread	GC 뒤 heap 미회복
Hikari	active-pending-max	pending 지속 증가
MySQL	query-lock-deadlock-I/O	lock과 p99 동시 증가
Redis	hit-miss-latency-eviction	hit 하락-eviction
Kafka	rate-lag-retry-E2E lag	부하 뒤 lag 미회복
비즈니스	재고-잔액-중복-유실	HTTP 성공과 최종 상태 불일치

회원 ID-주문 ID처럼 값이 계속 늘어나는 식별자를 metric label에 넣지 않습니다. 시계열 수가 폭증해 Prometheus 자체가 병목이 될 수 있습니다.

7. 프로젝트 전용 후속 실험

A. Hot SKU 재고 경합

같은 상품에 주문을 집중하고 비관적 잠금의 정확성과 대기시간을 함께 봅니다.

초기 재고 = 성공적으로 반영된 주문 수 + 최종 재고

확인: 성공 주문 수, 최종 재고, p95-p99, DB lock wait-deadlock, Hikari pending.

B. 포인트 동시 충전

한 회원에게 동시에 충전합니다. HTTP 성공 수만 세지 않고 lost update가 없는지 최종 잔액을 확인합니다.

최종 잔액 = 초기 잔액 + 성공 수 x 충전액

C. PG 지연과 회로 차단기

pg-simulator는 100-500ms 지연과 실패를 만듭니다. DB 트랜잭션 중 동기 외부 호출이 Hikari 연결을 오래 점유하는지 봅니다. fallback 때문에 HTTP 200이 실제 결제 성공을 뜻하지 않을 수 있어 DB 상태도 확인합니다.

D. 주문 -> Kafka -> streamer

HTTP p99와 이벤트 end-to-end 지연을 분리합니다. producer 오류, outbox 최고 대기 시간, consumer lag, 처리 완료 수를 확인합니다.

선행 gate: streamer 카운터 null 가능성, broker ack 전 outbox 완료 처리, 재고와 쿠폰의 서로 다른 트랜잭션 시점을 먼저 검증합니다.

내 실험 가설

변경할 변수 한 가지, 예상 결과, 반증 조건을 적으세요.

8. 실패 진단 클리닉

사례 1 - p99만 급증

- 성공 요청만의 p99인지 확인한다.
- k6와 서버 p99를 비교한다.
- Tomcat busy-Hikari pending-CPU-GC를 같은 시간축에서 본다.
- DB slow query-lock wait를 확인한다.
- 부하 단계와 캐시 만료 시점을 겹쳐 본다.

사례 2 - 목표 RPS 미달

- dropped_iterations를 확인한다.
- 부하 발생기 CPU-RAM 여유를 확인한다.
- preAllocatedVUs-maxVUs가 작은지 본다.
- 서버 연결 거부-빠른 실패를 확인한다.

사례 3 - Redis가 DB보다 느림

- 실제 cache hit인지 확인한다.
- key가 요청마다 달라지는지 확인한다.
- 직렬화-응답 크기를 비교한다.
- replica lag와 cold-warm 혼합 여부를 확인한다.

레이어 진단 문제

서버 p99와 Hikari pending은 증가했지만 CPU는 35%입니다. 다음에 수집할 증거 세 가지를 적으세요.

9. 실험 기록지

환경과 입력

항목	기록
실험 ID / Git commit	
시각 / 시간대	
CPU / 메모리	
Java / k6 / Docker	
데이터 건수 / 인덱스	
profile / variant	
목표 RPS / 실행 시간	
워밍업 제외 구간	

측정 결과

지표	결과
actual RPS	
p50 / p95 / p99	
오류 / check / dropped	
CPU / Tomcat busy	
Hikari active / pending	
DB lock / Redis / Kafka	
비즈니스 불변식	

가설 - 변경 - 결론

10. 종합 연습

1. 평균 80ms, p95 200ms, p99 2s입니다. 어떤 사용자가 문제를 겪습니까?

2. p99와 Hikari pending이 함께 증가하지만 CPU는 낮습니다. 첫 진단 대상은 무엇입니까?

3. Redis warm 실험에서 매 요청마다 브랜드 순서를 바꾸면 어떤 일이 생깁니까?

4. HTTP 오류율이 0%여도 주문 성능 실험이 실패할 수 있는 이유는 무엇입니까?

5. 목표 500 RPS, 실제 420 RPS이며 dropped가 증가합니다. 먼저 무엇을 구분해야 합니까?

11. 정답과 판단 근거

1. 상위 약 1%의 요청이 수초 지연을 겪는 tail latency 문제입니다. 평균에서 숨겨집니다.
2. DB 연결 획득 대기, 느린 쿼리, 긴 트랜잭션, DB 잠금을 먼저 확인합니다.
3. 현재 key에 브랜드 순서가 들어가므로 논리적으로 같은 조건도 다른 key가 되어 miss와 key 파편화가 늘어납니다.
4. 초과 판매, lost update, 중복 이벤트처럼 HTTP 200 뒤의 최종 상태가 틀릴 수 있기 때문입니다.
5. 서버 용량 부족과 부하 발생기 VU-CPU 부족을 먼저 구분합니다. k6와 서버 자원을 함께 봅니다.

전이 질문

상품 조회 p99는 개선됐지만 DB CPU가 크게 올랐습니다. 배포해야 할까요? 사용자 SLO, 최대 부하, 비용, 다른 쿼리에 미치는 영향을 함께 비교해 판단합니다.

12. 공식 자료와 범위

아래 자료는 프로토콜-도구 동작을 확인한 1차 자료입니다. 워크북의 시간 예시는 합성 예시이며, 실제 측정값은 9장 기록지에 조건과 함께 남깁니다.

- Prometheus - Histograms and summaries: prometheus.io/docs/practices/histograms/
- Prometheus - histogram_quantile:
prometheus.io/docs/prometheus/latest/querying/functions/#histogram_quantile
- Prometheus - Instrumentation: prometheus.io/docs/practices/instrumentation/
- Micrometer - Histograms and percentiles:
docs.micrometer.io/micrometer/reference/concepts/histogram-quantiles.html
- Spring Boot 3.4 - Metrics: docs.spring.io/spring-boot/3.4/reference/actuator/metrics.html
- Grafana k6 - Thresholds: grafana.com/docs/k6/latest/using-k6/thresholds/
- Grafana k6 - Open vs closed: grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/
- Grafana k6 - Prometheus remote write:
grafana.com/docs/k6/latest/results-output/real-time/prometheus-remote-write/
- Google SRE - Monitoring distributed systems: sre.google/sre-book/monitoring-distributed-systems/
- Google SRE - Service level objectives: sre.google/sre-book/service-level-objectives/

QA 체크

확인	상태
측정값과 합성 예시 구분	완료
p95-p99-히스토그램 설명	완료
실행 명령과 기대 신호	완료
실패 진단과 정답	완료
보안: Actuator 운영 노출 주의	운영에서는 인증-내부망 필요
생략 범위 공개	완료